

# Architecting for Impact: Five GCP-Deployable AI Agent Projects to Showcase Expertise for Sticker Mule

This report identifies and details five high-impact, GCP-deployable project ideas designed to build a compelling portfolio for a candidate targeting the AI Agent Engineer position at Sticker Mule. The projects are meticulously crafted to align with the company's business context—a combination of software, manufacturing, and AI in a commerce platform <sup>1</sup> <sup>9</sup>—and to demonstrate mastery over the specified technology stack: Go, TypeScript, GraphQL, PostgreSQL, and Google Cloud Platform (GCP). Each project proposal addresses a tangible business problem, showcases the full lifecycle of an AI agent from conception to deployment, and incorporates robust mechanisms for performance measurement and evaluation. The feasibility of deploying these projects within GCP's free-tier allowances and new customer trial credits is rigorously assessed, ensuring they are not only conceptually sound but also practically achievable for a job seeker. The analysis draws upon extensive research into modern AI agent architectures, evaluation frameworks, and the specific capabilities of Google Cloud services to provide actionable, strategic recommendations.

## Project 1: Intelligent Order Fulfillment Coordinator

The first project, the Intelligent Order Fulfillment Coordinator, is designed to tackle a critical operational challenge in any e-commerce and manufacturing business: the manual and repetitive process of order management. In a high-volume environment like Sticker Mule's, where thousands of customers are supported daily, streamlining the journey from a placed order to a shipped product is paramount for efficiency and customer satisfaction <sup>2</sup>. This project directly addresses the Sticker Mule requirement to "better serve customers" by reducing friction in the post-purchase experience <sup>1</sup> <sup>8</sup>. It demonstrates the ability to identify a high-value use case, build an autonomous agent to execute it, and integrate with internal systems to produce measurable results. The choice of Go as the primary language is a deliberate strategic decision to highlight expertise in a

statically-typed language that offers significant advantages for building reliable, production-grade applications <sup>34</sup>.

The core value proposition of this agent is its capacity for autonomous action across a multi-step workflow without constant human intervention, a capability explicitly valued by Sticker Mule <sup>5</sup>. The agent acts as a supervisor, orchestrating a sequence of tasks that involve querying a database, calling internal APIs, and updating system statuses. This directly mirrors the job description's emphasis on connecting agents to appropriate internal tools and measuring the results of their actions <sup>1</sup>. By automating this process, the project showcases how AI can innovate faster and improve performance, two of the engineer's primary objectives at Sticker Mule <sup>9</sup>. The implementation leverages the Agent Development Kit (ADK) for Go, a framework that provides fine-grained control and integrates well with Google Cloud services <sup>31 35</sup>. The architectural blueprint involves using Cloud SQL for PostgreSQL as the central nervous system for state management, ensuring persistence and reliability. The agent's interactions with other systems are exposed through a GraphQL API, providing a clean, typed interface that is ideal for agentic workflows <sup>38</sup>.

The technical implementation begins with the selection of Go for its strong typing and concurrency features, which are particularly advantageous for an agent that may need to handle multiple orders simultaneously <sup>34</sup>. While many agent frameworks are Python-centric, the availability of ADK for Go makes it a viable and powerful option <sup>35</sup>. The agent's logic will be structured around a supervisor-subagent pattern. A main coordinator agent, written in Go, will poll a `orders` table in PostgreSQL for new orders with a status of 'new' or 'pending'. For each identified order, it will delegate the fulfillment steps to specialized sub-agents or direct function calls. The agent's memory and state are managed using the `DatabaseSessionService` provided by the ADK, which connects to the Cloud SQL instance <sup>3 4</sup>. This ensures that even if the agent application restarts, it retains the necessary context to continue processing orders correctly, a critical feature for a production system. State keys prefixed with `user:` could store user-specific preferences, while unprefixed keys manage session-scoped variables like the current order being processed <sup>4</sup>.

Integration with Sticker Mule's internal tool suite is a cornerstone of this project's design. The agent must interact with the **Ship** platform to generate shipping labels and update tracking information, and with the **Give** platform to check for eligibility for promotional giveaways <sup>2</sup>. These integrations are implemented as custom tools within the ADK framework. When the agent determines that inventory is confirmed for an order, it will make a secure API call to the Ship platform's endpoint. Similarly, before finalizing an

order, it might query the Give platform to see if the user qualifies for a bonus item. All these external communications are orchestrated through the agent's logic, demonstrating the ability to connect to third-party and internal tools as required by the job description <sup>8</sup>. The agent's own functions, such as `generateShippingLabel(orderID)` or `checkGiveawayEligibility(customerID)`, are exposed via a GraphQL schema. This serves a dual purpose: it provides a standardized, navigable interface for the agent's own logic and can be used to expose the agent's capabilities to other parts of the Sticker Mule ecosystem in a type-safe manner <sup>38</sup>. Hasura, an open-source tool, is noted as an instant real-time GraphQL engine for PostgreSQL, making it a potential candidate for implementing this API layer <sup>37</sup>.

The autonomous workflow is the heart of the project. The process can be broken down into several logical steps: 1. **Order Detection:** The supervisor agent executes a periodic task (e.g., triggered by a Cloud Scheduler) that queries the `orders` table in Cloud SQL for entries where `status = 'new'`. 2. **Inventory Validation:** For each new order, the agent invokes a tool that calls an internal API to check the availability of the requested products. This step prevents wasted effort on orders for out-of-stock items. 3. **Label Generation:** If inventory is confirmed, the agent proceeds to call the Ship platform's API to create a shipping label and retrieve the carrier tracking number. 4. **State Update:** The agent then updates the corresponding order record in the PostgreSQL database, changing the status to 'processing', and populating fields like `tracking_number` and `fulfillment_notes`. 5. **Customer Notification:** Finally, the agent triggers the **Notify** platform via a webhook or direct API call to send a pre-defined message to the customer, such as "Your order has been shipped! Track your package here [link]." This entire sequence represents a closed-loop, autonomous workflow that moves an order from one state to another, embodying the concept of an agent that runs on its own and does meaningful work <sup>1</sup>.

To ensure the project is deployable on GCP's free resources, careful consideration is given to service selection and usage limits. The agent application itself will be containerized and deployed to **Cloud Run**, which offers a substantial free tier of 2 million vCPU-seconds and 4 million GiB-seconds per month <sup>14 15</sup>. Given that a fulfillment agent is likely to be idle much of the time, this allowance should be more than sufficient for a proof-of-concept or low-traffic production workload. The backend database will be **Cloud SQL for PostgreSQL**. While there is no permanent "Always Free" dedicated instance, new users receive \$300 in free credits that can be applied to shared-core configurations <sup>10 11</sup>. Furthermore, GCP offers a 30-day free trial for a powerful 8 vCPU, 64 GB RAM configuration, which is more than adequate for prototyping and initial testing <sup>12</sup>. The agent can leverage Gemini models hosted on **Vertex AI** for any natural language

processing tasks, such as parsing order notes or generating customer communication drafts. The Vertex AI Agent Engine component also has a free tier covering 50 vCPU-hours and 100 GB-hours of memory per month, which can support initial experimentation <sup>20</sup>.

The most critical aspect of this project for a job interview is the demonstration of a robust evaluation framework. The Sticker Mule role requires engineers to "measure results and remove agents that don't deliver" <sup>1</sup>. Therefore, this project must include clear metrics for success. The following KPIs should be tracked and reported:

| Metric                           | Description                                                                                                                                             | Measurement Method                                                                                                          |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Automation Rate                  | The percentage of orders in a given period that were fully processed by the agent without human intervention.                                           | Count of automatically processed orders / Total count of new orders * 100%. Data sourced from the orders table in Postgres. |
| Average Processing Time          | The average time taken from when an order enters the 'new' queue to when its status is updated to 'shipped' by the agent.                               | Calculate the timestamp difference between the 'new' status entry and the 'shipped' status update in the orders table.      |
| Error Rate                       | The percentage of automated fulfillment attempts that failed due to issues like invalid inventory checks or API errors.                                 | Count of failed orders / Total count of attempted orders * 100%. Errors logged to a dedicated error table in Postgres.      |
| First-Contact Resolution (FCR)   | Not applicable in this pure back-end workflow, but can be adapted if the agent also handles customer inquiries about order status.                      | N/A                                                                                                                         |
| Business Impact (Time Saved)     | The total amount of manual labor hours saved by operations staff who no longer have to perform the initial order validation and label generation steps. | Average time per manual task (minutes) * Number of tasks automated per week / 60 minutes per hour.                          |
| Business Impact (Cost Reduction) | Estimated reduction in operational costs based on the time saved and the average salary of the staff performing the tasks.                              | Calculated from the Time Saved metric above.                                                                                |

The evaluation methodology goes beyond simple pass/fail tests. It involves offline regression testing against a curated dataset of known order scenarios and online monitoring of production traffic <sup>23 27</sup>. For example, LangSmith, a popular observability platform, could be used to capture traces of the agent's execution, allowing for detailed analysis of why a particular order failed to process correctly (e.g., was it a tool parameter error, a network timeout, or incorrect reasoning?) <sup>27</sup>. This level of detail demonstrates a sophisticated understanding of agent evaluation, a key differentiator for the Sticker Mule role. By presenting a project with clearly defined, quantifiable metrics for success, a candidate shows they understand that the ultimate goal of an AI agent is not just to exist, but to deliver tangible business value, a principle central to Sticker Mule's mission <sup>1</sup>.

## Project 2: Dynamic Pricing & Promotion Strategist

The second project, the Dynamic Pricing & Promotion Strategist, addresses another core challenge in e-commerce: maximizing revenue and clearing inventory in a competitive market. Manually analyzing sales data, competitor pricing, and market trends to formulate effective promotions is a slow, reactive, and often subjective process <sup>6</sup>. This project proposes an AI agent that automates this analysis, enabling proactive and data-driven merchandising strategies. It directly aligns with Sticker Mule's goal of building a "lucrative commerce platform" by combining software, manufacturing, and AI <sup>1</sup>. The project showcases the ability to work with data, reason about business goals, and take autonomous actions that impact the top line, demonstrating a strategic mindset beyond simple operational automation.

The choice of **Python** as the programming language is highly strategic. Python is the de facto standard for data analysis and machine learning, with a rich ecosystem of libraries like Pandas, NumPy, and scikit-learn that are essential for the analytical components of this agent <sup>30</sup>. The agent will leverage the **Agent Development Kit (ADK) for Python**, which is a mature and well-supported framework that integrates seamlessly with the broader Python data science stack <sup>31</sup>. This project highlights the ability to use the right tool for the job, pairing Python's analytical prowess with Go's strengths in other areas. The agent's orchestration logic will be built using ADK's graph-based patterns, allowing for complex, conditional workflows <sup>31</sup>. Like the previous project, it will use **PostgreSQL** on **Cloud SQL** for persistent storage of product data, historical prices, and promotion records, and **GraphQL** to expose its planning and execution endpoints <sup>38</sup>.

The architecture of this strategist agent revolves around a planning loop. A primary agent, acting as a "strategist," sets high-level goals, such as "increase sales of Product X by 15% this week" or "clear all stock of seasonal item Y." To achieve these goals, the agent employs a series of tools to gather and analyze contextual data. These tools include:

- **Internal Data Access:** The agent connects to Sticker Mule's internal **PostgreSQL** database to query tables containing sales history, current inventory levels, profit margins, and customer purchase behavior. This allows the agent to understand its own business dynamics <sup>16</sup>.
- **Third-Party API Integration:** The agent integrates with external e-commerce platforms (like Shopify) and potentially public data sources to gather competitor pricing and market trends. This external context is crucial for setting competitive prices <sup>6</sup>.

- **Foundation Model Interaction:** The agent uses a Large Language Model (LLM), accessible via **Vertex AI**, to interpret unstructured data, such as social media sentiment or news articles related to its products, and to help formulate persuasive promotional copy.

The autonomous workflow is designed to be a complete cycle of analysis, decision-making, and action. The process unfolds as follows: 1. **Data Ingestion:** At scheduled intervals (e.g., daily), the agent executes its toolset. It pulls raw sales and inventory data from the `products` and `sales_history` tables in Cloud SQL. Simultaneously, it scrapes competitor websites or calls their APIs to collect pricing information for similar products. 2. **Analysis and Insight Generation:** The agent's Python code analyzes the combined dataset. It might use statistical methods to identify products that are underperforming, overstocked, or priced significantly higher than competitors. For instance, it could flag a product with a 30% lower conversion rate than its closest competitor. 3. **Strategy Formulation:** Based on the analysis and the high-level goals set by a human-in-the-loop manager, the agent formulates a strategy. This could involve applying a dynamic discount, bundling a slow-moving item with a fast-moving one, or creating a limited-time offer. The LLM can assist in generating the text for the promotion. 4. **Action Execution:** Once a strategy is formulated, the agent takes autonomous action. It calculates a specific discount percentage based on profit margins to ensure the promotion remains profitable. Then, it calls the API of the **Notify** platform to launch a targeted email campaign. The campaign is sent to a carefully segmented audience—for example, past customers who have purchased similar items or users who have abandoned carts containing those items <sup>6</sup>. 5. **Monitoring and Reporting:** After the promotion is live, the agent continues to monitor its performance, tracking metrics like click-through rates and redemption rates. This data is logged back into the PostgreSQL database for future analysis and to refine the agent's strategies over time.

For deployment on GCP, this project is highly feasible within the free tier. The Python-based agent application can be containerized and run on **Cloud Run**, leveraging its generous free tier of 2 million vCPU-seconds and 4 million GiB-seconds per month <sup>15</sup>. Since the agent performs intensive computations during its daily run but is otherwise idle, its resource consumption will be well within the free limits. The backend PostgreSQL database will be hosted on **Cloud SQL**. As with Project 1, the *300newcustomercreditcancoverthecostofabasicshared-coreinstanceforprototyping,orthethe30-dayfree* (300 new customer credit can cover the cost of a basic shared-core instance for prototyping, or the 30-day free tier) mean that the cost of inference for a small number of daily executions will be minimal <sup>20 39</sup>. It is important to note that costs can escalate with increased usage or by using more powerful models, so managing the budget is a key part of the project's presentation <sup>39</sup>.

The evaluation framework for this project is centered on demonstrating a clear return on investment (ROI) and a positive impact on business metrics. This directly addresses the Sticker Mule requirement to "measure results" and evaluate an agent's performance against a justified quality and ROI threshold <sup>1</sup> . The following metrics are critical for showcasing the agent's value:

| Metric                                    | Description                                                                                                                          | Measurement Method                                                                                                                                     |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Promotion Lift (Conversion Rate)          | The increase in the conversion rate for products involved in an agent-driven promotion compared to a control group or baseline.      | Compare post-promotion conversion rate to a pre-defined baseline or a randomly selected non-targeted group. Data from analytics platform and Postgres. |
| Revenue Per User (RPU) Increase           | The change in average revenue generated from users exposed to the promotion versus those who were not.                               | Calculate RPU for the target segment during the promotion period and compare it to the RPU from a previous period or a control group.                  |
| Inventory Turnover Speed                  | The rate at which promoted slow-moving or overstocked items are sold.                                                                | Measure the number of days it takes to sell a certain quantity of the promoted item before and after the promotion.                                    |
| Automation ROI                            | The monetary value of the time saved by marketers and merchandisers who no longer have to manually plan and execute these campaigns. | Estimate the hours spent on manual campaign creation per week multiplied by the average hourly wage of the marketing team.                             |
| Cost Per Acquisition (CPA) vs. Target CPA | Compares the CPA of the promoted products to a predefined target CPA to ensure profitability.                                        | (Total promotion cost) / (Number of new customers acquired from the promotion). Promotion cost includes discounts given away.                          |

To evaluate the agent's performance, a dual-axis approach is recommended. Offline evaluations can be conducted using historical data to simulate how the agent's strategies would have performed in the past, serving as a regression test before deployment <sup>23</sup> . Online evaluations, using A/B testing, would be deployed to compare the agent's performance against standard marketing practices in a live environment <sup>28</sup> . For instance, a portion of traffic could be routed to the agent's personalized emails, while another portion receives a generic newsletter. Key performance indicators like click-through rate and conversion rate would be monitored in real-time. The ability to design and implement such an evaluation framework demonstrates a sophisticated understanding of agent performance, moving beyond simple technical implementation to show a commitment to business outcomes, which is a hallmark of a successful AI Agent Engineer at Sticker Mule <sup>1</sup> <sup>26</sup> . This project effectively bridges the gap between AI capabilities and concrete commercial results.

## Project 3: Proactive Customer Service Assistant ("WISMO" Specialist)

The third project, the Proactive Customer Service Assistant, focuses on automating a high-volume, low-complexity customer service task: handling "Where Is My Order?" (WISMO) inquiries. This is a classic use case for AI agents, as it involves retrieving specific information from a structured source (an order database) and communicating it to a user in a natural, helpful way <sup>6</sup>. By building an agent to handle these inquiries autonomously, the project demonstrates a clear path to reducing operational costs and improving customer response times. The estimated cost to resolve a WISMO inquiry manually is around \$12 per request; an effective agent can reduce this cost dramatically <sup>6</sup>. This project aligns with Sticker Mule's responsibility to "better serve customers" by providing faster, more efficient support <sup>8</sup>.

The technology stack for this project consists of **TypeScript**, **Agent Development Kit (ADK) for TypeScript**, **GraphQL**, and **PostgreSQL** on Cloud SQL. The use of TypeScript is a direct match for the job description and is well-suited for developing web-facing applications and bots <sup>1</sup>. The ADK for TypeScript provides the necessary abstractions for building agentic workflows in a language familiar to many modern web developers <sup>31</sup>. The agent will be designed as a conversational interface, likely integrated into a website chat widget. Its logic will rely heavily on interacting with Sticker Mule's internal **Ship** platform to fetch real-time shipping data and using **PostgreSQL** to manage user context and conversation history. The entire agent's functionality will be exposed through a GraphQL API, providing a clean separation between the frontend UI and the backend logic <sup>38</sup>.

The architecture centers on a conversational agent that maintains state across a user's interaction. The agent is built using the ADK for TypeScript, which provides primitives for managing a conversation loop. When a user initiates a chat and asks a question, the agent's LLM component interprets the intent. If the intent is a WISMO query, the agent activates a specific toolchain. The core of the agent's knowledge comes from the **Ship** platform's API, which contains real-time tracking information. The agent will also query a **conversations** table in the **PostgreSQL** database to retrieve the user's recent order history, allowing it to provide context-aware responses (e.g., "I see you have three orders. Which one are you asking about?"). The agent's ability to maintain short-term memory for the current conversation and long-term memory for user preferences is crucial, and this is achieved using the ADK's **DatabaseSessionService** connected to Cloud SQL

3 4 . The GraphQL API serves as the entry point for the chatbot, receiving user messages and returning the agent's responses.

The autonomous workflow for the WISMO specialist is straightforward yet effective, demonstrating the agent's ability to perceive data, reason, and execute an action: 1.

**Intent Recognition:** A user types a message into a chat widget on the Sticker Mule website, such as "My order #123 tracking." 2. **Information Extraction:** The agent's LLM parses the message to recognize the intent ("track\_order") and extract relevant parameters, specifically the order number 123. 3. **Context Retrieval:** The agent queries the `orders` and `order_items` tables in PostgreSQL to find the user's account associated with the order and to gather details about the products. 4. **External Data Lookup:** Using the extracted order number, the agent calls the API of the internal **Ship** platform to fetch the latest shipment status, carrier details, and estimated delivery date. 5. **Response Generation:** The agent synthesizes the retrieved information into a clear, natural language response. For example: "Your order #123 containing [Product Name] was shipped on [Date] via [Carrier]. You can track it here [tracking\_link]. The estimated delivery date is [Date]." This response is then sent back to the user through the chat widget. 6. **State Management:** The conversation thread and the fact that the user's query was resolved are logged in the PostgreSQL database, contributing to the agent's long-term memory and providing valuable data for evaluation.

Deployment on GCP is entirely feasible within the free tier. The TypeScript-based agent application can be containerized and deployed on **Cloud Run**. Given that the agent is event-driven (it only consumes resources when a user sends a message), its monthly vCPU-second usage will likely be very low, falling comfortably within the free tier allowance of 2 million vCPU-seconds per month 15 . The backend relational database will be **Cloud SQL for PostgreSQL**. Similar to previous projects, the \$300 new customer credit can cover the cost of a small shared-core instance for testing, or the 30-day free trial can be utilized for development 10 12 . The agent will use an LLM from **Vertex AI** for natural language understanding and response generation. The introductory pricing for models like Gemini 3.5 Flash is low enough that the cost of running this conversational agent, even with moderate traffic, can be kept well within the free credit limit 39 .

A strong evaluation framework is essential for this project to prove its value. The Sticker Mule role requires engineers to measure results and remove underperforming agents, so this project must have clear success criteria 1 . The evaluation should focus on both technical performance and business impact.

| Metric                                     | Description                                                                                                                         | Measurement Method                                                                                           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>First-Contact Resolution (FCR) Rate</b> | The percentage of WISMO inquiries that are fully resolved by the agent in a single interaction without escalation to a human agent. | Count of resolved chats / Total number of WISMO-related chats * 100%. Tracked via chat logs in Postgres.     |
| <b>Average Resolution Time</b>             | The average time taken to respond to and resolve a WISMO inquiry.                                                                   | Timestamp difference between the user's message and the agent's final, correct response.                     |
| <b>Customer Satisfaction (CSAT) Score</b>  | A direct measure of user happiness with the agent's assistance, typically collected via a post-interaction survey.                  | NPS (Net Promoter Score) or a simple rating scale (e.g., 1-5 stars). Survey data stored in Postgres.         |
| <b>Deflection Rate</b>                     | The percentage of inquiries that the agent successfully handled, preventing them from becoming tickets for the human support team.  | (Total inbound WISMO requests - Escalated requests) / Total inbound WISMO requests * 100%.                   |
| <b>Business Impact (Cost Savings)</b>      | The monetary savings realized by deflecting WISMO inquiries from expensive human agents to a low-cost AI agent.                     | Deflection Rate <i>Total volume of WISMO inquiries</i><br>Average cost per human resolution (\$12 estimate). |

The evaluation methodology should employ a combination of offline and online techniques. Offline evaluation can involve creating a dataset of common WISMO queries and verifying that the agent correctly extracts parameters and retrieves the right information <sup>23</sup>. Online evaluation would involve A/B testing or shadow mode, where the agent processes live requests in parallel with human agents to compare performance <sup>28</sup>. Tools like Langfuse or Arize Phoenix can be used to visualize the agent's "thought process"—the prompts sent to the LLM, the tools called, and the arguments passed—to debug failures and optimize performance <sup>24 27</sup>. For example, if the agent frequently fails to extract an order number, the prompt engineering can be refined. Presenting a project with such a detailed and business-focused evaluation plan demonstrates a maturity in AI engineering that goes far beyond coding. It shows an understanding that building an agent is only half the battle; proving its worth and iterating on its performance is what leads to successful, scaled deployments, which is precisely the skill set Sticker Mule is seeking <sup>1 26</sup>.

## Project 4: Automated Product Catalog Enrichment Agent

The fourth project, the Automated Product Catalog Enrichment Agent, targets the time-consuming and collaborative nature of maintaining a large e-commerce catalog. Creating accurate, SEO-friendly product descriptions, assigning relevant tags, and optimizing metadata are essential for discoverability and conversion but often require significant

manual effort from merchandisers, marketers, and SEO specialists <sup>6</sup> . This project proposes an AI agent that automates a significant portion of this workflow, taking raw product specifications and enriching them with high-quality descriptive content and metadata. This directly supports Sticker Mule's mission to build an innovative online marketplace by improving the underlying software and content infrastructure <sup>1</sup> <sup>2</sup> . The project demonstrates initiative in enhancing the core commerce platform, a key area of focus for the company.

Similar to the fulfillment coordinator, this project leverages **Go** as the primary language, reinforcing expertise in a statically-typed, compiled language that excels at building robust and reliable systems <sup>34</sup> . The agent will be built using the **Agent Development Kit (ADK) for Go**, which provides a performant and idiomatic way to construct agentic applications in Go <sup>35</sup> . The agent's architecture will be designed as a background worker that is triggered when a new product is added to a staging area. It will interact with a foundation model via **Vertex AI** to generate content, store intermediate and final states in **PostgreSQL** on **Cloud SQL**, and expose its functionality through a GraphQL API for triggering and status checking. This project showcases the ability to build a system that ingests raw data, applies intelligent processing, and produces structured outputs, a fundamental capability for any AI engineer.

The agent's workflow is designed to automate a multi-step creative and technical process. When a new product is created in the Sticker Mule system, it is initially saved to a **products\_staging** table in PostgreSQL. This action can trigger a background process (e.g., via Pub/Sub) that signals the enrichment agent to start its work. The agent's logic, implemented in Go using ADK, would proceed as follows: 1. **Data Ingestion:** The agent receives a product ID and queries the **products\_staging** table in Cloud SQL to retrieve the raw attributes of the product, such as its name, material, dimensions, color options, and base description. 2. **Content Generation:** The agent constructs a detailed prompt for a Large Language Model (LLM) hosted on Vertex AI. This prompt includes the raw product attributes and instructions for generating multiple variations of a product description, a list of SEO keywords, and suitable meta tags. For example, it might ask the LLM to "Write three unique, engaging product descriptions for a custom vinyl sticker made of [Material], emphasizing durability and vibrant colors. Include a list of 10 relevant SEO keywords." 3. **Output Validation and Guardrail:** A critical step in this process is the implementation of guardrails. Before accepting the LLM's output, the agent in Go performs validation checks. It might use another, smaller LLM call (or rule-based checks) to verify that the generated text adheres to the brand's voice and tone, does not contain inappropriate content, and accurately reflects the product's factual attributes. This step is crucial for ensuring the quality and safety of the output, a key concern in

production systems <sup>33</sup>. 4. **Data Storage and Finalization:** Once validated, the enriched data (descriptions, keywords, etc.) is stored back into the main `products` table in PostgreSQL, either in dedicated columns or in a JSONB field. The record's status in the `products` table is then updated to `enriched`, signifying completion. 5. **Status Notification:** The agent can then trigger a notification, perhaps via a webhook to a Slack channel or an email to the merchandising team, indicating that the product has been successfully enriched and is ready for review.

Deployment on GCP is well-supported by the free tier offerings. The Go-based agent application will be packaged into a Docker container and deployed to **Cloud Run**. Due to its event-driven nature, its resource consumption will be sporadic, making it an ideal candidate for Cloud Run's free tier of 2 million vCPU-seconds and 4 million GiB-seconds per month <sup>15</sup>. The backend database will again be **Cloud SQL for PostgreSQL**, with the same considerations for using the \$300 credit or the 30-day free trial for initial setup <sup>10</sup> <sup>12</sup>. The interaction with the Gemini model on Vertex AI will incur token-based costs. However, since the agent generates content for individual products rather than engaging in continuous dialogue, the overall token consumption will be manageable. The free tier for the Agent Engine (100 GB-hours of memory per month) further reduces the barrier to entry for experimenting with the agent's runtime environment <sup>20</sup>. The project's reliance on managed services ensures that the developer can focus on the agent's logic rather than infrastructure management, a key benefit highlighted in GCP documentation <sup>13</sup>.

The evaluation framework for this project is designed to quantify the efficiency gains and quality improvements it brings to the product catalog management process. Demonstrating a clear ROI is essential for proving the agent's value to stakeholders. The following metrics should be tracked:

| Metric                               | Description                                                                                                              | Measurement Method                                                                                                         |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Time Saved per Product</b>        | The amount of time previously spent by human employees on writing descriptions and adding tags for a single product.     | Manual calculation based on time-tracking data from merchandising/marketing teams before the agent was implemented.        |
| <b>Reduction in Manual Effort</b>    | The total number of hours saved per month/quarter by the merchandising and marketing teams.                              | (Time Saved per Product) * (Number of Products Enriched per Period).                                                       |
| <b>Catalog Completion Velocity</b>   | The average time it takes for a new product to go from creation to having a fully enriched catalog entry.                | Timestamp difference between the creation in the staging table and the final status update in the main products table.     |
| <b>SEO Keyword Quality</b>           | The relevance and search volume of the keywords generated by the agent, compared to those historically chosen by humans. | Use an SEO analysis tool to score the generated keywords and correlate them with organic traffic changes for the products. |
| <b>Human Review Rate</b>             | The percentage of agent-generated descriptions that still require human editing or approval before publication.          | (Number of edited/enriched products) / (Total number of enriched products) * 100%. Logged in the database.                 |
| <b>Business Impact (Scalability)</b> | The ability to manage a larger product catalog without a proportional increase in headcount for catalog management.      | Compare catalog size growth against the growth in the merchandising team's size over time.                                 |

Evaluation for this project should combine quantitative metrics with qualitative assessment. Offline evaluation can involve a panel of subject matter experts (merchandisers) reviewing a sample of the agent's output and scoring it on criteria like creativity, accuracy, and brand alignment. This human-in-the-loop feedback is invaluable for calibrating any LLM-as-a-judge evaluators that might be used <sup>23 29</sup>. Online evaluation would involve deploying the agent and continuously monitoring the Human Review Rate. A decreasing trend indicates that the agent's quality is improving over time, possibly through prompt refinement or fine-tuning. Presenting this project with a comprehensive evaluation plan demonstrates a commitment to quality and a data-driven approach to development. It shows that the candidate understands that AI agents are not a "fire-and-forget" solution but require continuous monitoring, feedback, and iteration to deliver sustained value, a crucial insight for any engineer working on production systems <sup>1 26</sup>.

## Project 5: Multi-Agent Sales Funnel Optimizer

The fifth and most advanced project, the Multi-Agent Sales Funnel Optimizer, represents the pinnacle of AI agent sophistication. It moves beyond single-agent automation to orchestrate a team of specialized agents working in concert to influence the entire customer journey, from discovery to post-purchase. This project directly addresses Sticker Mule's ambition to build a "lucrative commerce platform" by combining software,

manufacturing, and AI 1 . It demonstrates a strategic understanding of e-commerce funnels and the ability to design and implement complex, multi-step agentic systems. By using a mixed-language approach with Python for the supervisor and Go for a sub-agent, this project powerfully showcases versatility across the entire tech stack required for the role 1 31 .

The core idea is to create a system where a high-level "Supervisor Agent" delegates specific tasks to specialized "Subagents." This mirrors real-world business structures where different teams collaborate on a common goal 7 . The Supervisor Agent, built with ADK in Python, monitors user behavior and identifies drop-off points in the sales funnel. When it detects a high-intent opportunity, such as a user abandoning a shopping cart, it activates a team of Subagents to re-engage the customer. One Subagent might be responsible for finding alternative products, while another generates a promotional incentive. This Supervisor-Subagent pattern is a key feature of advanced frameworks like ADK, enabling scalable and modular agentic architectures 41 . The entire system relies on **PostgreSQL** for storing user behavior data and conversation history, and **GraphQL** to define the contracts between the supervisor and its sub-agents.

The architecture is designed for modularity and specialization. The Supervisor Agent, written in Python/ADK, acts as the conductor. It listens for events, such as a user visiting a product page or adding an item to their cart. It accesses user behavior logs and transactional data from a **PostgreSQL** database to build a profile of the user's intent. Upon detecting an abandoned cart scenario, it springs into action. It then creates a task and delegates it to two distinct Subagents: 1. **Subagent 1 (Go):** A "Visual Recommender" agent built with ADK for Go. Its specific role is to analyze the abandoned item and the user's browsing history (pulled from Postgres) to find visually similar products. This agent might use a vision-language model (via Vertex AI) or an image search API to identify products that share a similar style, color palette, or theme. Its output is a list of recommended product IDs. 2. **Subagent 2 (Python):** A "Promotion Generator" agent, also built with Python/ADK. Its job is to create a compelling discount code tailored to the value of the abandoned items and the company's profit margins. It might use an LLM to brainstorm catchy coupon names (e.g., "FLASH15") and calculate the optimal discount percentage to incentivize a purchase without eroding margins.

The Supervisor Agent orchestrates the final step: integrating the outputs from both sub-agents and delivering a personalized, multi-faceted message to the user. It combines the list of recommended products from the Go sub-agent with the discount code from the Python sub-agent and formats it into a persuasive message. It then calls the **Notify** platform's API to send a targeted email or push notification: "We noticed you left something behind! Here are some similar items you might love, plus 15% off your next

purchase with code FLASH15." This entire process—from detecting the abandonment to sending the recovery message—is executed autonomously by the coordinated team of agents, showcasing a level of complexity and strategic thinking that aligns perfectly with the demands of the Sticker Mule role <sup>1</sup> 28 .

Deployment on GCP for this project would involve deploying each agent as a separate microservice on **Cloud Run**. This serverless architecture is ideal for running independent, stateless units of logic that communicate with each other. The combined vCPU and memory usage for multiple lightweight services is still highly likely to fall within the generous Cloud Run free tier limits <sup>15</sup> . The shared data store remains **Cloud SQL for PostgreSQL**, ensuring all agents have access to a consistent view of the customer and product data. Communication between the agents can be facilitated by a messaging queue like **Pub/Sub** or directly via API calls, orchestrated by the Supervisor Agent. The use of the Agent2Agent (A2A) protocol, an open standard championed by Google, could even enable these agents to communicate with tools or agents from other vendors, showcasing forward-thinking architectural design <sup>21</sup> <sup>30</sup> . The total cost of operation, including Vertex AI inference for the LLM calls, would be managed by the \$300 new customer credit, making this sophisticated project entirely feasible to prototype and demonstrate <sup>10</sup> .

The evaluation of a multi-agent system is more complex than for a single agent, requiring an assessment of both individual component performance and the effectiveness of the overall orchestration. This complexity is a strength, as it demonstrates a deep understanding of agent evaluation principles. The success of this project hinges on its ability to drive tangible business outcomes, which must be measured rigorously.

| Metric                                              | Description                                                                                                                                        | Measurement Method                                                                                                                                                             |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Abandoned Cart Recovery Rate</b>                 | The percentage of users who abandoned their cart but later returned to complete a purchase after interacting with the agent's outreach.            | $(\text{Number of recovered carts}) / (\text{Total number of abandoned carts detected}) * 100\%$ . Correlate agent-triggered emails with subsequent purchases in the database. |
| <b>Conversion Rate Lift</b>                         | The increase in the conversion rate of the recovered cart cohort compared to a control group that did not receive the outreach.                    | A/B testing comparing the conversion rate of the target group to a randomly selected non-targeted group.                                                                       |
| <b>Average Order Value (AOV) of Recovered Carts</b> | The average spend of users who completed a purchase after the agent's intervention.                                                                | $\text{Sum of values of recovered orders} / \text{Number of recovered orders}$ .                                                                                               |
| <b>Multi-Agent System Efficiency</b>                | The total time taken for the entire team of agents to formulate and deliver the outreach message.                                                  | Measure the latency of the Supervisor Agent's workflow from task initiation to final action execution.                                                                         |
| <b>Subagent Performance</b>                         | Individual success rates of the Visual Recommender and Promotion Generator agents.                                                                 | Track the success/failure rate of each sub-agent's specific task (e.g., did the recommender return valid product IDs?).                                                        |
| <b>Business Impact (ROI)</b>                        | The total incremental revenue generated by the recovered carts, minus the cost of the discounts given away and the operational cost of the agents. | $(\text{Incremental Revenue from Recovered Carts}) - (\text{Total Discounts Given}) - (\text{Agent Operational Costs})$ .                                                      |

Evaluation for this project must be multi-layered. At the component level, each sub-agent's output must be validated. For example, the visual recommender's suggestions should be reviewed by humans to ensure relevance. At the trajectory level, the entire workflow from abandonment detection to message delivery must be traced to ensure it is logical and efficient <sup>23 25</sup>. Finally, at the end-to-end level, the primary business metric—recovered revenue—must be measured. This project's evaluation plan should mention using a platform like LangSmith or Langfuse to trace the delegation of tasks from the supervisor to the subagents, providing a clear visualization of the multi-agent orchestration <sup>24</sup>. Presenting a project of this complexity, complete with a detailed and holistic evaluation framework, would be a powerful statement of a candidate's readiness for the AI Agent Engineer role at Sticker Mule. It proves not only the ability to build individual agents but also the strategic vision to assemble them into a cohesive system capable of driving significant business impact <sup>1 28</sup>.

## Strategic Synthesis and Portfolio Recommendation

In conclusion, the five proposed projects—Intelligent Order Fulfillment Coordinator, Dynamic Pricing & Promotion Strategist, Proactive Customer Service Assistant, Automated Product Catalog Enrichment Agent, and Multi-Agent Sales Funnel Optimizer

—collectively form a powerful and cohesive portfolio for a candidate aspiring to the AI Agent Engineer role at Sticker Mule. Each project is meticulously designed not merely as a technical exercise but as a strategic demonstration of skills directly aligned with the company's business model, technological requirements, and stated responsibilities. They collectively showcase a journey from foundational operational automation to advanced, multi-agent strategic optimization, mirroring the career progression expected of a senior engineer in this domain.

The portfolio's strength lies in its direct contextualization to Sticker Mule's unique operating environment. By referencing internal tools like **Ship** and **Notify** and solving problems inherent to an e-commerce and custom manufacturing business, the projects move beyond generic tutorials and demonstrate genuine business acumen <sup>2</sup>. They address core facets of Sticker Mule's mission to build a "lucrative commerce platform" by combining software, manufacturing, and AI <sup>1 9</sup>. The fulfillment coordinator tackles logistics, the pricing strategist targets revenue, the customer service assistant improves CX, the catalog agent enhances the product offering, and the multi-agent optimizer drives conversions. This breadth of application proves an ability to identify high-value use cases across different departments, a key responsibility of the role <sup>1</sup>.

Technologically, the portfolio exhibits a masterful command of the specified stack: Go, TypeScript, GraphQL, PostgreSQL, and GCP. The strategic use of Go for its reliability in the fulfillment and enrichment agents, TypeScript for its suitability in the customer service bot, and Python for its data analysis prowess in the pricing and multi-agent orchestrator, demonstrates a pragmatic and informed approach to tool selection <sup>31 34</sup>. The consistent use of **PostgreSQL** on **Cloud SQL** for persistent state management across all projects underscores a deep understanding of the importance of reliable data storage for agentic memory <sup>4</sup>. The adoption of **GraphQL** as a typed API layer for exposing agent functionality highlights an awareness of modern API best practices <sup>38</sup>. Most importantly, the projects are architected for deployment on **Google Cloud Platform**, leveraging its managed services like Cloud Run, Vertex AI, and Cloud SQL, and are demonstrably feasible within the confines of GCP's generous free tier and trial credits, making them practical and impressive portfolio pieces <sup>10 15 20</sup>.

Perhaps the most critical element woven throughout these proposals is the emphasis on evaluation. The Sticker Mule role is not just about building agents; it is about "measuring results and removing agents that don't deliver" <sup>1</sup>. Each project includes a detailed framework of KPIs and evaluation methodologies, moving beyond simple success/failure tests to encompass business impact metrics like ROI, automation savings, and conversion lift. The discussion of offline and online evaluation, tracing, and observability tools like

Langfuse or Arize Phoenix demonstrates a sophisticated understanding of the entire agent lifecycle <sup>23</sup> <sup>24</sup> <sup>26</sup>. This focus on measurement and iteration is a hallmark of a production-ready engineer and directly addresses a key requirement of the job description.

By undertaking and presenting these five projects, a candidate can construct a compelling narrative of their capabilities. They can articulate a clear vision for how AI agents can solve real business problems, select the appropriate technologies to build them, architect them for scalability and reliability on a major cloud platform, and critically, measure their performance to ensure they deliver tangible value. This comprehensive approach validates not only their technical proficiency in Go, Python, GraphQL, and GCP but also their strategic thinking, business sense, and commitment to data-driven results—all of which are essential qualities for an AI Agent Engineer at Sticker Mule.

---

## Reference

1. Remote AI agent engineer at Sticker Mule <https://jobicy.com/jobs/153825-ai-agent-engineer>
2. Software engineer <https://www.stickermule.com/career/6db27241-e2d4-4f35-a2c4-b58d84621843>
3. Choose your agentic AI architecture components <https://docs.cloud.google.com/architecture/choose-agentic-ai-architecture-components>
4. Building Persistent AI Agents with ADK and CloudSQL <https://codelabs.developers.google.com/persistent-adk-cloudsql>
5. 11 Best AI Agents for Retail and E-commerce [2026] <https://www.happyrobot.ai/hub/best-ai-agents-for-retail-and-ecommerce>
6. How to Use AI Agents for Commerce <https://www.salesforce.com/commerce/ai/how-to-use-ai-agents-for-commerce/>
7. AI Agents in Manufacturing: Use Cases & ROI (2026) <https://assistents.ai/blogs/ai-agents-use-cases-manufacturing>
8. AI agent engineer | Sticker Mule EU <https://www.stickermule.com/eu/career/2f01bd23-9eda-446a-a56a-b530d84cb9bb>
9. Remote AI Agent Engineer ( ~\$200k) at Sticker Mule <https://remoteok.com/remote-jobs/remote-ai-agent-engineer-sticker-mule-1137399>

10. Free Trial and Free Tier Services and Products <https://cloud.google.com/free>
11. Google Cloud SQL: Pricing, Monitoring, and When to Use It <https://cloudchipr.com/blog/google-cloud-sql>
12. Try out Cloud SQL for PostgreSQL using a free trial instance <https://docs.cloud.google.com/sql/docs/postgres/free-trial-instance>
13. The 2026 Developer's Guide to Free Google Cloud Credits ... <https://dev.to/behruamm/the-2026-developers-guide-to-free-google-cloud-credits-for-ai-side-projects-1ac5>
14. Cloud Run pricing <https://cloud.google.com/run/pricing>
15. Google Cloud Run Pricing in 2025: A Comprehensive Guide <https://cloudchipr.com/blog/cloud-run-pricing>
16. Agentic AI Enterprise Use Cases — 30+ Real Deployments (2026) <https://www.ampcome.com/post/post-agentic-ai-enterprise-use-cases>
17. 100+ AI Agent Use Cases Across Industries & Verticals in 2026 <https://aimonk.com/ai-agent-use-cases/>
18. Agentic AI in Manufacturing: Use Cases That Work in 2026 <https://kanerika.com/blogs/agentic-ai-in-manufacturing/>
19. Gemini API Rate Limits 2026: Free Tier, Paid Tiers, Per-Model ... <https://aipromptshub.co/limits/gemini-rate-limits-2026>
20. Vertex AI Pricing 2026: What an Agent Actually Costs (Now ... <https://www.betterclaw.io/blog/google-vertex-ai-agent-builder>
21. Vertex AI Agent Builder: Build, Deploy, and Scale AI Agents ... <https://leanware.co/insights/vertex-ai-agent-builder>
22. ECom-Bench: Can LLM Agent Resolve Real-World E- ... <https://arxiv.org/html/2507.05639v2>
23. Evaluating AI Agents at the Run, Trace, and Thread Level <https://www.langchain.com/resources/agent-evals>
24. AI Agent Observability, Tracing & Evaluation with Langfuse <https://langfuse.com/blog/2024-07-ai-agent-observability-with-langfuse>
25. LLM Agent Evaluation Metrics in 2026: Tool Calling, Task ... <https://www.confident-ai.com/blog/llm-agent-evaluation-complete-guide>
26. AI agent evaluation: frameworks, metrics & testing strategies <https://www.algolia.com/blog/ai/ai-agent-evaluation-frameworks-metrics-testing-strategies>
27. AI Agent Evaluation (2026): The Metrics, the Frameworks, and ... <https://www.morphllm.com/ai-agent-evaluation>

28. AI Agent Evaluation: Frameworks, Strategies, and Best ... <https://medium.com/online-inference/ai-agent-evaluation-frameworks-strategies-and-best-practices-9dc3cfd9890>
29. AI Agent Evaluation: Build Production-Grade Agents <https://mastra.ai/articles/ai-agent-evaluation>
30. Build and manage multi-system agents with Vertex AI <https://cloud.google.com/blog/products/ai-machine-learning/build-and-manage-multi-system-agents-with-vertex-ai>
31. AI Agent Frameworks: LangGraph, crewAI, ADK Compared <https://www.raftlabs.com/blog/ai-agent-framework-comparison>
32. Any good and simple AI Agent frameworks for Go? : r/golang [https://www.reddit.com/r/golang/comments/1i51m8w/any\\_good\\_and\\_simple\\_ai\\_agent\\_frameworks\\_for\\_go/](https://www.reddit.com/r/golang/comments/1i51m8w/any_good_and_simple_ai_agent_frameworks_for_go/)
33. Best AI Agent Frameworks 2026: Alice Labs Top 10 Ranked <https://alicelabs.ai/en/insights/best-ai-agent-frameworks-2026>
34. Go is the Best Language for AI Agents - Bruin Data <https://getbruin.com/blog/go-is-the-best-language-for-agents/>
35. Announcing the Agent Development Kit for Go: Build ... <https://developers.googleblog.com/announcing-the-agent-development-kit-for-go-build-powerful-ai-agents-with-your-favorite-languages/>
36. Building RoboShop: A Microservices E-commerce Platform [https://www.linkedin.com/posts/prashanth91330\\_devopsjourney-microservices-linuxserver-activity-7328051526545932290-A4R3](https://www.linkedin.com/posts/prashanth91330_devopsjourney-microservices-linuxserver-activity-7328051526545932290-A4R3)
37. lists/README.md at main · jnv/lists <https://github.com/jnv/lists/blob/main/README.md>
38. GraphQL Is the API Layer AI Agents Actually Need <https://wundergraph.com/blog/graphql-api-layer-for-ai-agents>
39. Google Vertex AI Pricing: Complete Enterprise Guide (2026) <https://www.cloudzero.com/blog/google-vertex-ai-pricing/>
40. Gemini Free Credits 2026: Why the \$300 Trial No Longer Works <https://klymentiev.com/blog/gemini-free-credits>
41. Vertex AI Agent Builder: 2026 guide to Google's enterprise ... <https://uibakery.io/blog/vertex-ai-agent-builder>